CY Cergy-Paris Université

RAPPORT

pour le projet de Génie Logiciel
Licence 2 Informatique, 2ème Année

sur le sujet

Simulation de Missions Spatiales

rédigé par

Lydia LACHEMOT, Thomas LENGLIN,
Sabri GHOUDANE



Avril 2026

Table des matières

1 Introduction **4**
1.1 Contexte du projet . . . 4
1.2 Objectif du projet . . . 4
1.3 Organisation du rapport . . . 4

2 Spécification du projet **5**
2.1 Notions de base et contraintes du projet . . . 5
2.1.1 Fonctionnement général du logiciel . . . 5
2.1.2 Notions et terminologies de base . . . 5
2.1.3 Contraintes et limitations connues . . . 6
2.2 Fonctionnalités attendues du projet . . . 7

3 Conception et mise en œuvre **10**
3.1 Architecture globale du logiciel . . . 10
3.2 Classes de données . . . 11
3.3 IHM Graphique . . . 13
3.4 Conception des traitements (processus) . . . 14
3.5 Design Patterns utilisés . . . 20

4 Manuel utilisateur **21**
4.1 Settings . . . 21
4.2 Simulation . . . 23

5 Déroulement du projet **26**
5.1 Réalisation du projet par étapes . . . 26
5.2 Répartition des tâches, communication et intégration . . . 28

6 Conclusion et perspectives **29**
6.1 Résumé du travail réalisé . . . 29
6.2 Améliorations possibles du projet . . . 30

Table des figures

1 Diagramme des modules fonctionnels du logiciel . . . 10
2 Diagramme des classes de données physiques . . . 11
3 Diagramme des classes de données des entités dynamiques . . . 12
4 Enchaînement entre les fenêtres de l'interface graphique . . . 13
5 Maquette de la fenêtre de configuration . . . 14
6 Maquette de la fenêtre de simulation . . . 15
7 Vue d'ensemble des patrons de conception utilisés . . . 20
8 Page d'ouverture de notre programme . . . 21
9 Bouton "Help" dans la barre de menu des deux pages . . . 21
10 Bouton "Launch Simulation" dans la barre de menu de "Settings" . . . 22
11 Liste déroulante du type de mission . . . 22
12 Liste déroulante de la destination . . . 22
13 Champ textuel personnalisable . . . 22
14 Zone de texte récupérant les valeurs de propulsion . . . 23
15 Page principale du programme de la simulation . . . 23
16 Boutons du menu de la page "Simulation" . . . 24
17 Boutons de gestions de la simulation . . . 24
18 Boutons de gestion de l'affichage . . . 24
19 Champ d'informations en temps réel de la simulation . . . 25

20 Chaine d'indication de l'avancement de la simulation 25
21 Graphes des valeurs de vitesse,carburant et d'altitude de la fusée 25
22 Répartition de l'avancement du projet par version 26

Liste des tableaux

1 Fonctionnalités attendues du logiciel 7
2 Angle de virage gravitationnel en fonction de l'altitude 9
3 Répartition des tâches entre les membres de l'équipe 28

Remerciements

Nous remercions en premier lieu notre enseignant encadrant, Monsieur Tianxiao LIU, pour la qualité de son enseignement en Génie Logiciel, sa disponibilité et ses conseils avisés tout au long de ce projet. Ses orientations méthodologiques et ses retours constructifs nous ont guidés dans nos choix de conception et nous ont permis d'approfondir notre compréhension des patrons de conception, de l'architecture logicielle et des bonnes pratiques de développement orienté objet. Nous tenons également à souligner la valeur des projets exemples fournis tout au long du semestre, qui nous ont servi de référence et d'inspiration pour structurer notre propre application, tant sur le plan de l'organisation du code que sur celui de la conception de l'interface graphique et de l'architecture générale du logiciel. Ses cours nous ont notamment permis de maîtriser des concepts fondamentaux tels que les design patterns (Singleton, Builder, Factory, Strategy), la séparation des responsabilités entre modules, ainsi que la conception d'interfaces graphiques en Java Swing. Ces enseignements ont directement influencé les décisions architecturales de notre simulateur spatial, depuis l'organisation en packages jusqu'à la mise en œuvre des managers et des builders.

Nous remercions également l'Université CY Cergy-Paris pour les ressources pédagogiques, les outils de développement et les infrastructures informatiques mis à notre disposition, qui ont rendu possible la réalisation de ce simulateur spatial en Java.

Enfin, nous remercions les auteurs des ressources en ligne et des articles scientifiques consultés, notamment les équipes de la NASA pour la documentation sur les missions Apollo et Mars Science Laboratory, ainsi que les contributeurs des ressources pédagogiques sur la mécanique spatiale qui ont nourri notre compréhension du domaine.

1 Introduction

1.1 Contexte du projet

La simulation spatiale est un domaine clé pour comprendre et anticiper les comportements des objets en vol dans l'espace. Dans le cadre de ce projet de Génie Logiciel, nous avons développé une application à partir du language de programmation Java permettant de simuler le lancement de fusées et le déroulement de missions spatiales. Ce simulateur modélise les lois physiques gouvernant la propulsion, la trajectoire et la mise en orbite de charges utiles telles que des satellites ou des sondes interplanétaires.

1.2 Objectif du projet

L'objectif principal de ce projet est de concevoir et implémenter un simulateur pédagogique de missions spatiales. Le simulateur permet à l'utilisateur de configurer une mission parmi trois types : orbite basse terrestre (LEO), orbite géostationnaire (GEO) et mission interplanétaire vers la Lune ou d'autres planètes du système solaire. Des valeurs de propulsion par défaut sont prédéfinies pour chaque type de mission, mais l'utilisateur peut les modifier s'il le souhaite. Une fois le lancement déclenché, la simulation se déroule de façon autonome en appliquant les lois physiques de la mécanique spatiale. L'utilisateur dispose uniquement de contrôles d'affichage tels que le zoom, la mise en pause, le suivi de la fusée et la vitesse de simulation, sans intervenir sur le comportement physique de la fusée. Sur le plan logiciel, le projet vise à produire une application robuste, maintenable et extensible, en appliquant les bonnes pratiques de conception orientée objet étudiées en cours de Génie Logiciel.

1.3 Organisation du rapport

Ce rapport est organisé en quatre sections principales. La section 2 présente les spécifications du projet à travers le cahier des charges et les exigences fonctionnelles et techniques. La section 3 détaille la réalisation technique, organisée autour de la modélisation des données, de l'interface graphique et des algorithmes de simulation physique.La section 4 constitue le manuel utilisateur et décrit les fonctionnalités accessibles depuis chaque fenêtre de l'application.La section 5 décrit le déroulement du projet, la répartition des tâches au sein de l'équipe et l'évolution du travail au fil des séances. Enfin, la section 6 conclut le rapport en dressant un bilan du travail accompli et en présentant les perspectives d'amélioration envisagées.

2 Spécification du projet

Dans cette section, nous présentons la spécification de notre logiciel réalisé. Ceci correspond principalement au document de spécification du projet (cahier des charges).

2.1 Notions de base et contraintes du projet

2.1.1 Fonctionnement général du logiciel

Le logiciel est une simulation spatiale 2D dans laquelle chaque objet possède une position, une vitesse, une accélération et une masse [tpe15]. L'objectif est de fournir une représentation pédagogique et réaliste du lancement d'une fusée depuis la Terre. L'utilisateur configure sa mission depuis un tableau de bord, puis lance la simulation qui se déroule de façon autonome en temps réel. La fusée effectue son décollage, consomme son carburant, sépare ses étages successivement, puis déploie sa charge utile selon la mission choisie : mise en orbite d'un satellite ou envoi d'une sonde interplanétaire.

La simulation est calculée à chaque pas de temps en appliquant les lois de la mécanique newtonienne, une approche également utilisée dans [LV05] pour la reconstitution de trajectoires réelles.

Le déroulement d'une mission suit cinq phases successives.

- `Première phase` : le décollage vertical , le moteur du premier étage s'allume, la poussée dépasse le poids de la fusée et celle-ci commence à monter verticalement.
- `Deuxième phase` : le virage gravitationnel, à partir d'une certaine altitude, la fusée incline progressivement sa trajectoire pour acquérir une composante horizontale de vitesse.
- `Troisième phase` : la séparation des étages, lorsqu'un étage a consommé tout son carburant ou atteint son altitude de séparation, il est éjecté et le moteur de l'étage suivant s'allume.
- `Quatrième phase` : la mise en orbite ou l'injection interplanétaire, la fusée atteint la vitesse orbitale et déploie sa charge utile.
- `Cinquième phase` : la poursuite de mission autonome, le satellite ou la sonde poursuit seul sa trajectoire orbitale.

2.1.2 Notions et terminologies de base

Fusée : Moyen de transport permettant d'amener un satellite ou une sonde dans l'espace. Elle est modélisée par plusieurs systèmes : un système structural, un système de propulsion (moteur et carburant), une charge utile (sonde ou satellite) et un système de guidage assurant la stabilité et le contrôle de la trajectoire. Elle est représentée par deux ou trois étages se succédant jusqu'à la mise en orbite.

Étage : Composant structurel de la fusée possédant son propre moteur et son propre réservoir de carburant. Lorsque le carburant d'un étage est épuisé ou qu'une altitude de séparation prédéfinie est atteinte, l'étage est largué afin d'alléger la fusée et d'améliorer son rendement. La masse de la fusée diminue ainsi progressivement au fil des séparations.

Charge utile : Partie de la fusée transportant l'objet à déployer dans l'espace. Dans ce projet, la charge utile est soit un satellite artificiel pour les missions LEO et GEO, soit une sonde spatiale pour la mission interplanétaire. Elle est libérée lors de la phase finale de la mission.

Satellite : Objet spatial tournant autour d'un corps plus massif. Dans ce projet, le satellite est lié à la mission choisie et est détaché lors de la phase de séparation des étages. Deux types d'orbites sont simulés : l'orbite basse terrestre (LEO) et l'orbite géostationnaire (GEO).

Sonde spatiale : Véhicule spatial envoyé dans l'espace par une fusée pour explorer une autre planète. Elle est utilisée dans le cadre de la mission interplanétaire et poursuit seule sa mission une fois déployée.

Orbite : Trajectoire circulaire d'un objet autour d'une planète. L'orbite basse (LEO) se situe à une altitude d'environ 200 000 m, tandis que l'orbite géostationnaire (GEO) se situe à environ 35 786 000 m.

Système solaire : Environnement spatial dans lequel se déroule la simulation. Il est composé de

planètes, chacune caractérisée par sa masse, son rayon et sa position. Dans ce projet, les planètes restent fixes durant la simulation, leur mouvement orbital autour du Soleil n'étant pas modélisé.

Mission spatiale : Opération planifiée consistant à envoyer une fusée pour mettre un satellite en orbite ou envoyer une sonde vers une planète. Trois missions sont disponibles dans ce logiciel : LEO, GEO et interplanétaire.

Temps réel : Le programme évolue au même rythme que le temps du monde réel. Le système met à jour en continu les données spécifiques telles que la vitesse, l'altitude et la masse de carburant restante, plus précisément les positions de chaque objet à chaque pas de temps.

Pas de temps : Intervalle de temps élémentaire utilisé pour la simulation. Dans ce projet, le pas de temps est fixé à $\Delta t = 1$ s, ce qui signifie que les positions, vitesses et accélérations de tous les objets sont recalculées à chaque seconde simulée. Ce pas de temps constant garantit la stabilité de l'intégration numérique d'Euler.

Trajectoire : Suite ordonnée des positions successives occupées par la fusée au cours de son vol. Dans ce projet, la trajectoire est calculée numériquement à chaque pas de temps par intégration des équations du mouvement, et est affichée graphiquement sous forme d'une courbe de points dans la fenêtre de simulation.

Virage gravitationnel : Méthode standard utilisée par les vraies fusées [ker24] pour économiser du carburant. La fusée passe progressivement d'un vol vertical à un vol horizontal en diminuant son angle d'inclinaison au fur et à mesure de l'altitude.

Propulsion : Système assurant la mise en mouvement de la fusée. Dans ce projet, la propulsion de chaque étage est composée d'un moteur à réaction caractérisé par sa poussée et son impulsion spécifique, et d'un réservoir contenant la masse de carburant. Le moteur consomme le carburant à chaque pas de temps, ce qui réduit progressivement la masse totale de la fusée et augmente son accélération.

Impulsion spécifique : Grandeur physique caractérisant l'efficacité d'un moteur de fusée. Elle représente la quantité de poussée produite par unité de masse de carburant consommée par seconde. Plus l'impulsion spécifique est élevée, plus le moteur est efficace. Elle est exprimée en secondes et notée I_{sp}.

Vecteur de mouvement : Ensemble des grandeurs cinématiques décrivant l'état de déplacement d'un objet à un instant donné. Dans ce projet, le vecteur de mouvement de chaque objet mobile est composé de sa vitesse sur l'axe x (v_x), sa vitesse sur l'axe y (v_y), son accélération sur l'axe x (a_x) et son accélération sur l'axe y (a_y).

Altitude : Distance verticale séparant un objet de la surface de la planète de référence. Dans ce projet, l'altitude est calculée comme la différence entre la distance euclidienne entre la fusée et le centre de la planète, et le rayon de cette planète :

$$h = d(fusee, planete) - R_{planete} \tag{1}$$

L'altitude est utilisée pour déterminer les conditions de séparation des étages, le calcul de la densité atmosphérique et l'ajustement de l'angle de virage gravitationnel.

2.1.3 Contraintes et limitations connues

La simulation est limitée à un environnement en deux dimensions. Les planètes du système solaire sont présentes dans le modèle de données mais restent fixes durant toute la simulation, leur mouvement orbital autour du Soleil n'étant pas modélisé. La physique est simplifiée : seules trois forces sont prises en compte lors du vol, à savoir la poussée des moteurs, la gravitation et la traînée atmosphérique. Le site de lancement est fixé au Centre Spatial Kennedy (latitude 28,586°).

Outils de développement :

1. Langage de programmation : Java
2. IDE : Eclipse
3. Outil de rédaction : Latex
4. Outil de gestion de version et de partage du code : GitHub

2.2 Fonctionnalités attendues du projet

Le tableau 1 présente les principales fonctionnalités attendues du logiciel, telles que définies dans le cahier des charges.

Fonctionnalité	**Description**
Tableau de bord	Au lancement de l'application, l'utilisateur dispose d'un tableau de bord depuis lequel il configure et lance sa mission
Choix de mission	Trois missions disponibles : LEO (orbite basse), GEO (orbite géostationnaire) et interplanétaire. L'utilisateur peut également nommer et décrire sa mission
Choix de destination	L'utilisateur sélectionne la planète de destination parmi une liste proposée
Paramétrage de la propulsion	L'utilisateur peut modifier la poussée et la masse de carburant de chaque étage. Des valeurs par défaut garantissant le succès de la mission sont prédéfinies
Décollage automatique	La fusée effectue son décollage en plusieurs étapes : mise en poussée, montée verticale, consommation du carburant, virage gravitationnel, puis éjection progressive des étages
Contrôle du temps	L'utilisateur peut accélérer, ralentir ou mettre en pause la simulation à tout moment
Zoom	L'utilisateur peut agrandir ou rétrécir l'affichage pour suivre la fusée ou visualiser l'ensemble du parcours
Informations en temps réel	Affichage continu de la vitesse, l'accélération, les étages restants, la masse, la position, la distance à la Terre et la durée de mission
Mise en orbite	Selon la mission, la fusée se place en orbite autour de la planète cible et déploie sa charge utile. La vitesse orbitale est calculée selon l'équation 12 de la vitesse orbitale
Satellisation	Pour les missions LEO et GEO, le satellite est éjecté à la vitesse orbitale correspondante et poursuit seul sa trajectoire
Mission interplanétaire	La sonde est transportée depuis la Terre vers la planète cible. La mission réussit si la vitesse de libération est atteinte ,définie en 14

TABLE 1 – Fonctionnalités attendues du logiciel

Comme illustré dans le tableau 1, le logiciel couvre l'ensemble du cycle d'une mission spatiale, depuis la configuration jusqu'au déploiement de la charge utile. L'utilisateur influence la réussite de la mission par le choix de ses paramètres de propulsion : des paramètres inadaptés conduisent à un échec de mission, les valeurs par défaut garantissent le succès.

Le calcul du décollage repose sur trois forces modélisées mathématiquement. La force de poussée exercée par le moteur est calculée selon l'équation 2, la force gravitationnelle selon l'équation 5, et la traînée atmosphérique selon l'équation 7. Ces trois forces sont combinées via la seconde loi de Newton 9 pour calculer la trajectoire à chaque pas de temps.

Force de poussée. Elle est exercée par le moteur de la fusée selon :

$$F_p = \dot{m} \cdot v_e \tag{2}$$

où $\dot{m}$ est le débit massique des gaz éjectés et v_e la vitesse d'éjection, définie par :

$$v_e = I_{sp} \cdot g_0 \tag{3}$$

avec I_{sp} l'impulsion spécifique du moteur et $g_0 = 9{,}81$ m/s^2.

Force du poids. La gravitation attire la fusée vers la planète centrale :

$$P = m \cdot g \tag{4}$$

où m est la masse de la fusée, qui diminue au fur et à mesure de la consommation du carburant.

Gravitation universelle. La force gravitationnelle exercée par une planète sur la fusée est calculée selon :

$$F_g = \frac{G \cdot M \cdot m}{d^2} \tag{5}$$

où $G = 6{,}674 \times 10^{-11}\ \mathrm{m^3 \cdot kg^{-1} \cdot s^{-2}}$ est la constante gravitationnelle universelle, M la masse de la planète, m la masse de la fusée et d la distance entre les deux objets. Cette force est décomposée sur les axes x et y :

$$F_{g_x} = -F_g \cdot \frac{\Delta x}{d} \quad ; \quad F_{g_y} = -F_g \cdot \frac{\Delta y}{d} \tag{6}$$

Force de traînée atmosphérique. Elle s'oppose au mouvement de la fusée :

$$F_f = \frac{1}{2} \cdot \rho \cdot C_d \cdot A \cdot v^2 \tag{7}$$

où ρ est la densité de l'air, C_d le coefficient de traînée, A la surface frontale de la fusée et v sa vitesse. La densité atmosphérique diminue exponentiellement avec l'altitude selon :

$$\rho(h) = \rho_0 \cdot e^{-h/8500} \tag{8}$$

où $\rho_0 = 1{,}225\ \mathrm{kg/m^3}$ est la densité au niveau de la mer.

Calcul de la trajectoire. La trajectoire est obtenue en appliquant la seconde loi de Newton à chaque pas de temps [LV05] :

$$\vec{a} = \frac{\vec{F}_{résultante}}{m} \tag{9}$$

La résultante est décomposée sur les axes x et y selon l'angle de la fusée.
La vitesse puis la position sont intégrées numériquement à chaque pas de temps Δt selon la méthode d'intégration d'Euler :

$$v_x(t + \Delta t) = v_x(t) + a_x \cdot \Delta t \quad ; \quad v_y(t + \Delta t) = v_y(t) + a_y \cdot \Delta t \tag{10}$$

$$x(t + \Delta t) = x(t) + v_x \cdot \Delta t \quad ; \quad y(t + \Delta t) = y(t) + v_y \cdot \Delta t \tag{11}$$

Vitesse orbitale. La vitesse nécessaire à la mise en orbite est donnée par [tpe15] :

$$v_{orb} = \sqrt{\frac{G \cdot M}{R}} \tag{12}$$

où R est la distance entre l'objet et le centre de la planète. Pour la Terre, cette vitesse est d'environ 7,9 km/s, et pour la Lune d'environ 1,7 km/s.

La distance entre deux objets est calculée à chaque pas de temps selon la formule euclidienne :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{13}$$

Cette distance est utilisée pour le calcul de la force gravitationnelle 5, le calcul de l'altitude 1 et la détermination de la mise en orbite.

Vitesse de libération. La vitesse minimale pour échapper à l'attraction gravitationnelle d'une planète est :

$$v_{lib} = \sqrt{\frac{2 \cdot G \cdot M}{d}} \tag{14}$$

Cette vitesse est vérifiée à chaque instant pour déterminer si la sonde a atteint la vitesse suffisante lors de la mission interplanétaire.

La consommation de carburant est mise à jour à chaque pas de temps selon la relation suivante, dérivée de l'équation de poussée 2 :

$$\dot{m} = \frac{F_p}{v_e} = \frac{F_p}{I_{sp} \cdot g_0} \tag{15}$$

La nouvelle masse de propergol après un pas de temps Δt est alors :

$$m_{propergol}(t + \Delta t) = m_{propergol}(t) - \dot{m} \cdot \Delta t \tag{16}$$

Angle de virage gravitationnel. L'angle d'inclinaison de la fusée est ajusté progressivement en fonction de l'altitude selon le tableau 2.

Altitude (m)	**Angle (°)**
$h < 1\,000$	90
$1\,000 \leq h < 5\,000$	85
$5\,000 \leq h < 10\,000$	80
$10\,000 \leq h < 20\,000$	75
$20\,000 \leq h < 30\,000$	70
$30\,000 \leq h < 40\,000$	60
$40\,000 \leq h < 60\,000$	50
$60\,000 \leq h < 80\,000$	40
$80\,000 \leq h < 100\,000$	30
$h \geq 100\,000$	10

TABLE 2 – Angle de virage gravitationnel en fonction de l'altitude

Comme illustré dans le tableau 2, la fusée passe progressivement d'un vol vertical à un vol quasi-horizontal à mesure qu'elle gagne en altitude, ce qui lui permet d'acquérir la vitesse horizontale nécessaire à la mise en orbite.

Trajectoire de transfert de Hohmann [Wik26]

Le transfert interplanétaire est modélisé par une ellipse reliant deux orbites circulaires de rayons r_1 et r_2.

Le demi-grand axe de l'orbite de transfert est donné par :

$$a = \frac{r_1 + r_2}{2} \tag{17}$$

La vitesse nécessaire au point de départ est donnée par :

$$v_{transfert} = \sqrt{\mu \left(\frac{2}{r_1} - \frac{1}{a} \right)} \tag{18}$$

avec :

— $\mu = G \cdot M$ le paramètre gravitationnel de la planète centrale
— r_1 le rayon de l'orbite initiale
— r_2 le rayon de l'orbite cible

La direction de la vitesse est tangentielle à l'orbite circulaire :

$$\vec{t} = \left(-\frac{y}{r}, \frac{x}{r} \right) \tag{19}$$

3 Conception et mise en œuvre

Dans cette section, nous décrivons la conception et la mise en œuvre technique du simulateur, en distinguant pour chaque composant les choix de conception effectués en amont et leur implémentation concrète en Java.

3.1 Architecture globale du logiciel

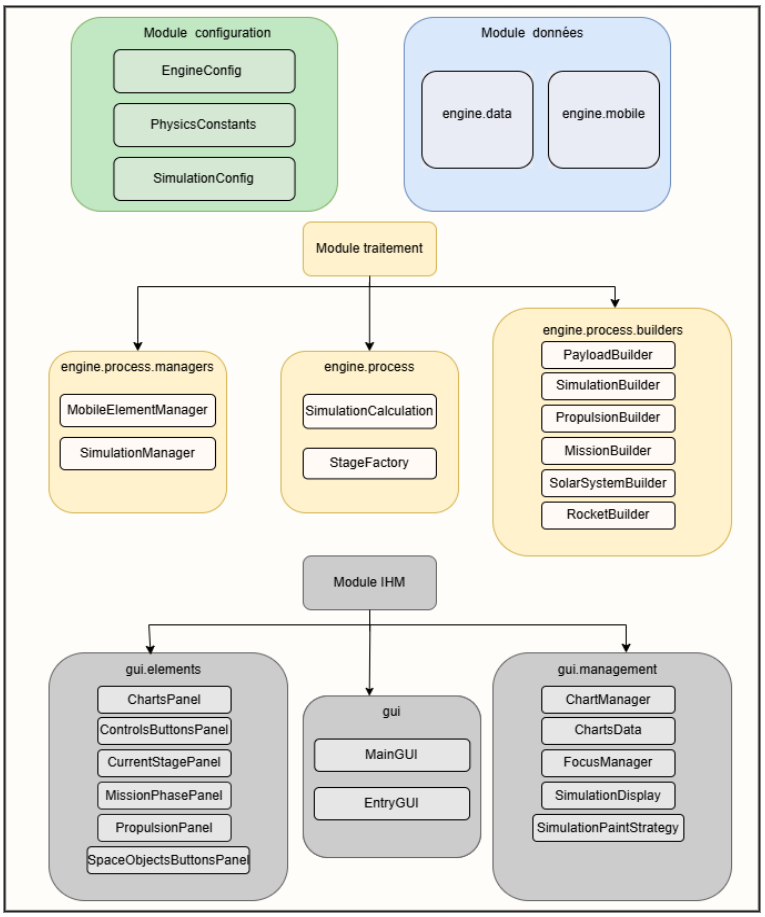


FIGURE 1 – Diagramme des modules fonctionnels du logiciel

Le logiciel est organisé en quatre modules fonctionnels aux responsabilités bien définies, comme illustré dans la figure 1.

Module configuration. Ce module regroupe les classes de constantes et de paramètres utilisés par l'ensemble de l'application. La classe `EngineConfig` définit les paramètres propres au moteur et aux missions, `PhysicsConstants` centralise les constantes physiques telles que la constante gravitationnelle et la densité de l'air, et `SimulationConfig` regroupe les paramètres d'affichage et de simulation.

Module données. Ce module est composé de deux packages. Le package `engine.data` regroupe les classes représentant les entités métier du domaine spatial : la mission, la propulsion, le moteur et le réservoir de carburant. Le package `engine.mobile` regroupe les objets en mouvement dans la simulation : la fusée, les étages, les satellites, la sonde, les planètes et les orbites.

Module traitement. Ce module est le cœur logique de la simulation. Il est composé de trois packages. Le package `engine.process.managers` contient `SimulationManager` qui orchestre l'ensemble de la simulation, et `MobileElementManager` qui gère les mises à jour physiques des objets mobiles à chaque pas de temps. Le package `engine.process` contient `SimulationCalculation` qui regroupe tous

les calculs de mécanique spatiale, et `StageFactory` qui centralise la création des étages. Le package `engine.process.builders` regroupe les builders responsables de la construction des objets complexes : `RocketBuilder`, `MissionBuilder`, `PropulsionBuilder`, `SimulationBuilder`, `SolarSystemBuilder` et `PayloadBuilder`.

Module IHM. Ce module regroupe l'ensemble des composants graphiques de l'interface utilisateur. Il est composé de trois packages. Le package `gui` contient les deux fenêtres principales : `EntryGUI` pour la configuration de la mission et `MainGUI` pour l'affichage de la simulation. Le package `gui.elements` regroupe les panneaux fonctionnels de l'interface : `ChartsPanel`, `ControlsButtonsPanel`, `CurrentStagePanel`, `MissionPhasePanel`, `PropulsionPanel` et `SpaceObjectsButtonsPanel`. Le package `gui.management` regroupe les classes de gestion de l'affichage graphique : `SimulationDisplay`, `SimulationPaintStrategy`, `FocusManager`, `ChartManager` et `ChartsData`.

Comme illustré dans la figure 1, les dépendances entre modules suivent un sens unique : le module IHM dépend du module traitement, qui lui-même dépend des modules données et configuration. Cette organisation garantit que chaque module peut évoluer indépendamment des autres et facilite la maintenance du logiciel.

3.2 Classes de données

Conception et mise en œuvre des classes de données pour le package data.

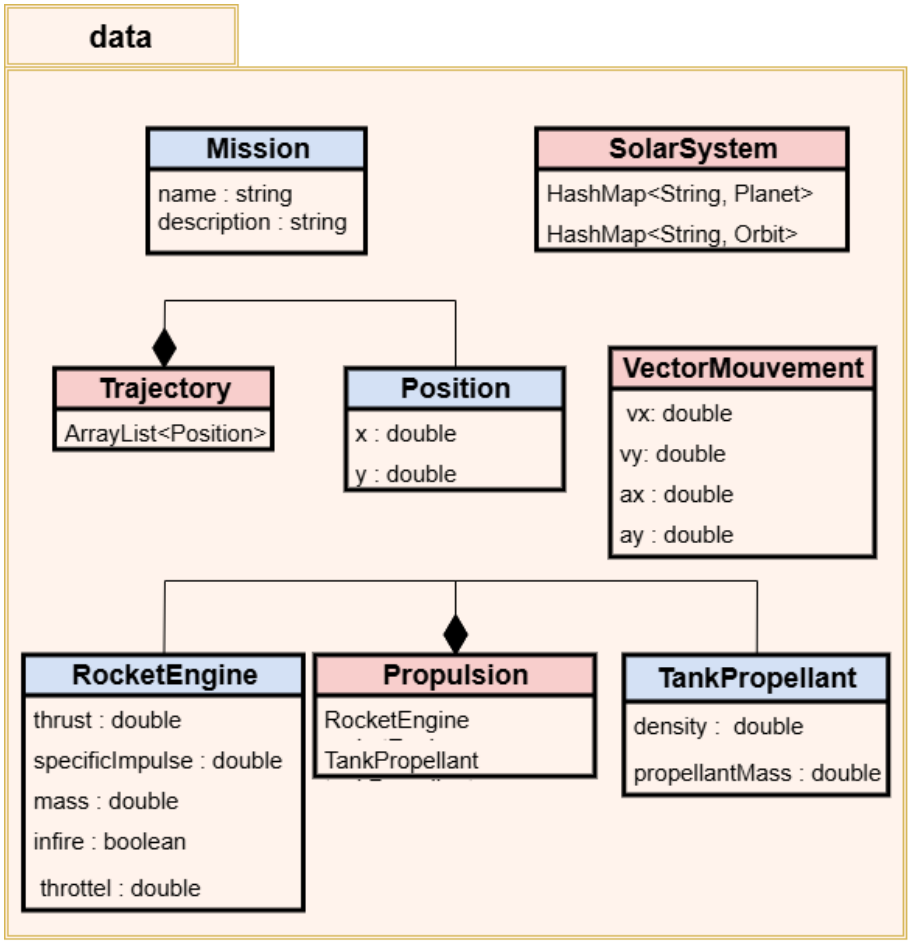


FIGURE 2 – Diagramme des classes de données physiques

Package data : Notre moteur est composé de deux packages de données, le premier nommé *data* contient les classes de données n'ayant pas de mouvement et représentant les valeurs algébrique de notre programme.

— `Mission` : récupère les informations textuelles inscrites et choisies par l'utilisateur.
— `SolarSystem` : singleton servant à contenir toutes les planètes et orbites du système.
— `Trajectory` : récupère les positions antérieurs de l'engin spatial pour construire sa trajectoire.
— `Position` : classe primordiale récupérant la position d'un objet dans la simulation.
— `VectorMouvement` : les coordonnées des vecteurs vitesse et accélération y sont stockés.
— `RocketEngine` : le moteur d'un étage, il contient toutes les valeurs physiques de la fusée nécessaire au fonctionnement du moteur.
— `Propulsion` : récupère le moteur et le carburant d'un étage.
— `TankPropellant` : le carburant spécifique à un étage.

Conception et mise en œuvre des classes de données pour le package mobile.

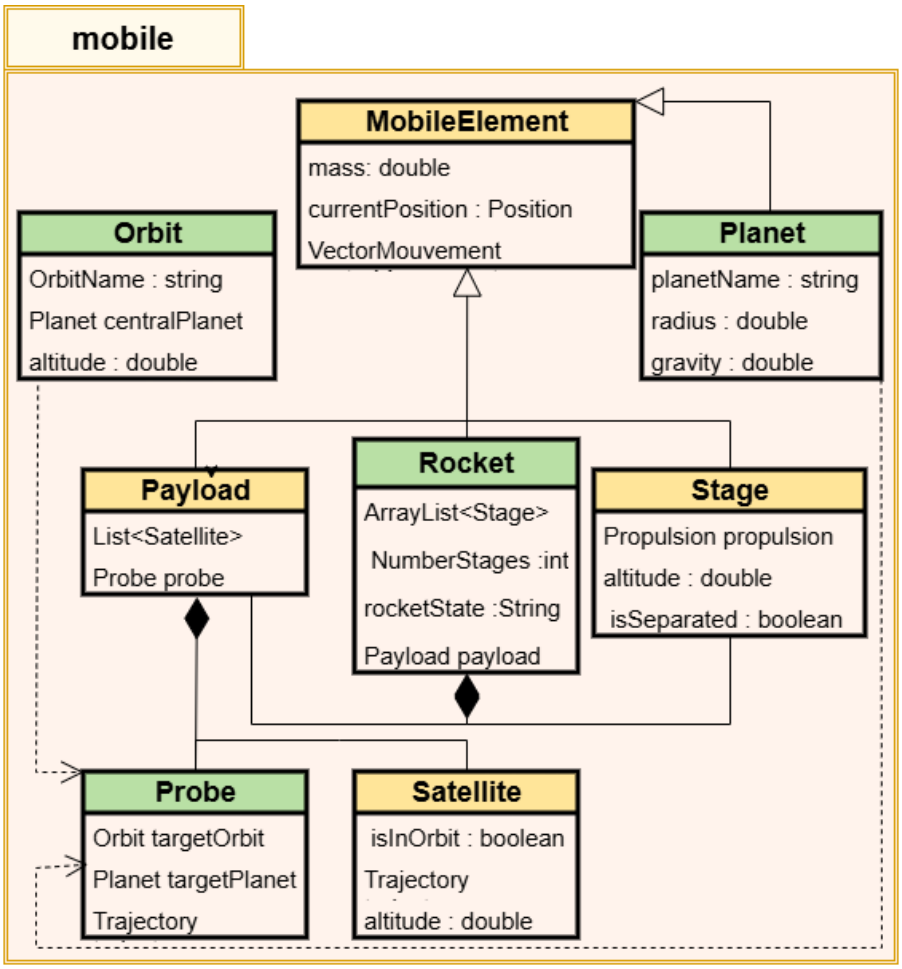


FIGURE 3 – Diagramme des classes de données des entités dynamiques

Package mobile. Le deuxième package nommé *mobile* contient les classes de données ayant un mouvement et sont toutes représentés graphiquement dans la simulation.
— `MobileElement` : classe mère du package, elle récupère la masse , les coordonnées du vecteur position, vitesse et l'accélération d'un objet.
— `Orbit` : l'orbite d'une planète est dépendante d'une planète en son centre, fixe dans la simulation.
— `Planet` : Constitue les valeurs physiques d'une planète.
— `Rocket` : Contient la charge utile et les étages, c'est une classe importante de notre programme.
— `Payload` : La charge utile qui contient soit une sonde soit des satellites.
— `Stage` : un étage de la fusée, il contient la propulsion venant du package data.
— `Probe` : Element de la charge utile qu'on envoie vers les autres planète, elle possède une orbite à atteindre ainsi qu'une planète.
— `Satellite` : Element de la charge utile qu'on envoie vers une orbite de la planète Terre

3.3 IHM Graphique

Dans cette sous-section, nous décrivons la conception et la mise en œuvre de l'interface graphique du simulateur

Conception de l'enchaînement des fenêtres. L'interface graphique a été conçue autour de deux fenêtres principales s'enchaînant séquentiellement. Ce choix reflète les deux phases distinctes du cycle d'utilisation : la phase de configuration et la phase de simulation. La séparation en deux fenêtres distinctes permet de ne charger les ressources de simulation qu'au moment du lancement, et non dès le démarrage de l'application.

Mise en œuvre de l'enchaînement. Le passage d'une fenêtre à l'autre est déclenché par le bouton `Launch Simulation` dans `EntryGUI`. Lorsque l'utilisateur clique sur ce bouton, `EntryGUI` récupère les paramètres saisis, appelle `SimulationBuilder.buildSimulation()` pour construire l'ensemble de la simulation, instancie `MainGUI` avec le `SimulationManager` créé, puis se ferme via `dispose()`.

Comme illustré dans la figure 4, ce mécanisme garantit qu'une seule fenêtre est active à la fois.

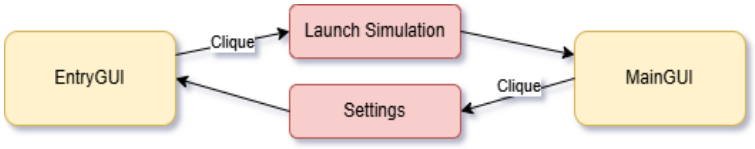


FIGURE 4 – Enchaînement entre les fenêtres de l'interface graphique

Conception de la fenêtre de configuration. La fenêtre de configuration `EntryGUI` a été conçue pour permettre à l'utilisateur de paramétrer sa mission avant le lancement. Elle est organisée en plusieurs zones distinctes : une zone de configuration de la mission, une zone de paramétrage de la propulsion et une zone d'affichage du schéma de la fusée. Cette organisation permet à l'utilisateur de visualiser en temps réel le schéma de la fusée correspondant au type de mission sélectionné.

Mise en œuvre de la fenêtre de configuration. La classe `EntryGUI` étend `JFrame` et utilise un `BorderLayout` comme gestionnaire de mise en page. Elle est composée des éléments suivants :

- Un panneau supérieur contenant le bouton `Launch Simulation`
- Un panneau central `SchemaPanel` affichant le schéma technique de la fusée sur fond quadrillé, avec les étages et la charge utile identifiés. Ce panneau surcharge la méthode `paintComponent()` pour dessiner l'image dynamiquement selon le type de mission sélectionné
- Un panneau de droite `PropulsionPanel` permettant de saisir la poussée et la masse de carburant pour chaque étage. Des valeurs par défaut sont prédéfinies pour garantir le succès de la mission
- Un panneau inférieur contenant les `JComboBox` pour le type de mission et la destination, ainsi que les `JTextField` pour le nom et la description de la mission
- Un menu `Help` permettant d'afficher une notice explicative

Maquette de la fenêtre de configuration. La figure 5 illustre l'organisation de la fenêtre de configuration `EntryGUI`. On y distingue le schéma technique de la fusée au centre, le panneau de propulsion à droite et la barre de configuration de mission en bas.

Conception de la fenêtre de simulation. La fenêtre de simulation `MainGUI` a été conçue pour offrir à l'utilisateur une vision complète de la mission en temps réel. Elle regroupe l'animation spatiale, les données physiques de la fusée et les contrôles d'affichage. La conception sépare clairement la zone d'animation centrale des zones d'informations et de contrôle disposées en périphérie.

Mise en œuvre de la fenêtre de simulation. La classe `MainGUI` étend `JFrame` et implémente `Runnable` pour exécuter la boucle de simulation dans un thread dédié. Elle est composée des éléments suivants :

- Un panneau supérieur contenant le bouton `Mission Settings`

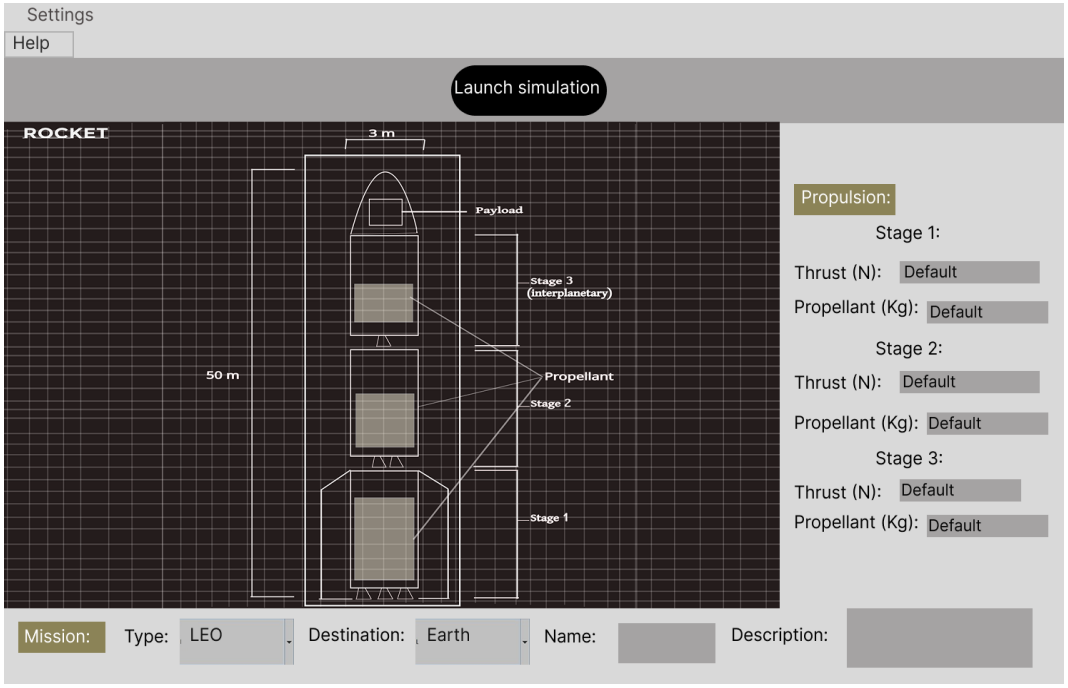


FIGURE 5 – Maquette de la fenêtre de configuration

— Un panneau central `SimulationDisplay` affichant l'animation spatiale en temps réel : la Terre, les orbites, la fusée, les étages séparés, les satellites et la sonde. Ce panneau délègue le rendu à `SimulationPaintStrategy`
— Un panneau de droite `eastPanel` regroupant : le panneau `stateRocketPanel` affichant l'état de la fusée, l'altitude, les étages restants et la durée de mission, le panneau `MissionPhasePanel` affichant la phase courante de la mission, le panneau `ControlsButtonsPanel` contenant les boutons `Launch`, `Pause/Continue`, `Reset`, `Speed Up`, `Slow Down` et le panneau `SpaceObjectsButtonsPanel` contenant les boutons `Zoom In`, `Zoom Out`, `Follow`, `Target Planet` et `Departure Planet`
— Un panneau inférieur `ChartsPanel` affichant en temps réel les graphiques de carburant, de vitesse et d'accélération

Maquette de la fenêtre de simulation. La figure 6 illustre l'organisation de la fenêtre de simulation `MainGUI`. On y distingue l'animation spatiale au centre, les boutons de contrôle en haut à droite, les informations de la fusée au milieu à droite, le panneau `MissionPhasePanel` en bas à droite et les graphiques de vitesse, carburant et accélération en bas.

3.4 Conception des traitements (processus)

Conception des managers. Les traitements ont été conçus autour de deux managers aux responsabilités clairement séparées. `SimulationManager` joue le rôle de coordinateur global : il possède le système solaire, le manager des mobiles, la mission et les propulsions. `MobileElementManager` implémente l'interface `MobileInterface` et est responsable uniquement des mises à jour physiques des objets mobiles à chaque pas de temps. Ce découplage permet de substituer facilement `MobileElementManager` par une autre implémentation sans modifier le reste de l'application.

Rôle de l'interface `MobileInterface`. L'interface `MobileInterface` définit le contrat que doit respecter tout manager de mobiles. Elle déclare l'ensemble des opérations du cycle de vie d'une mission : `rocketLaunch()`, `updateRocket()`, `updateRocketMass()`, `updateCurrentStage()`, `stageSeparation()`, `payloadSeparation()`, `orbitInsertion()`, `updateStage()`, `updateSatellite()`, `updateProbe()`, ainsi que les trois méthodes de tour `nextRoundLEO()`, `nextRoundGEO()` et `nextRoundInterplanatery()`. Ce découplage par interface permet à `SimulationManager` de ne dépendre que de l'abstraction `MobileInterface`, rendant `MobileElementManager` substituable par toute autre implémentation sans modifier le reste de

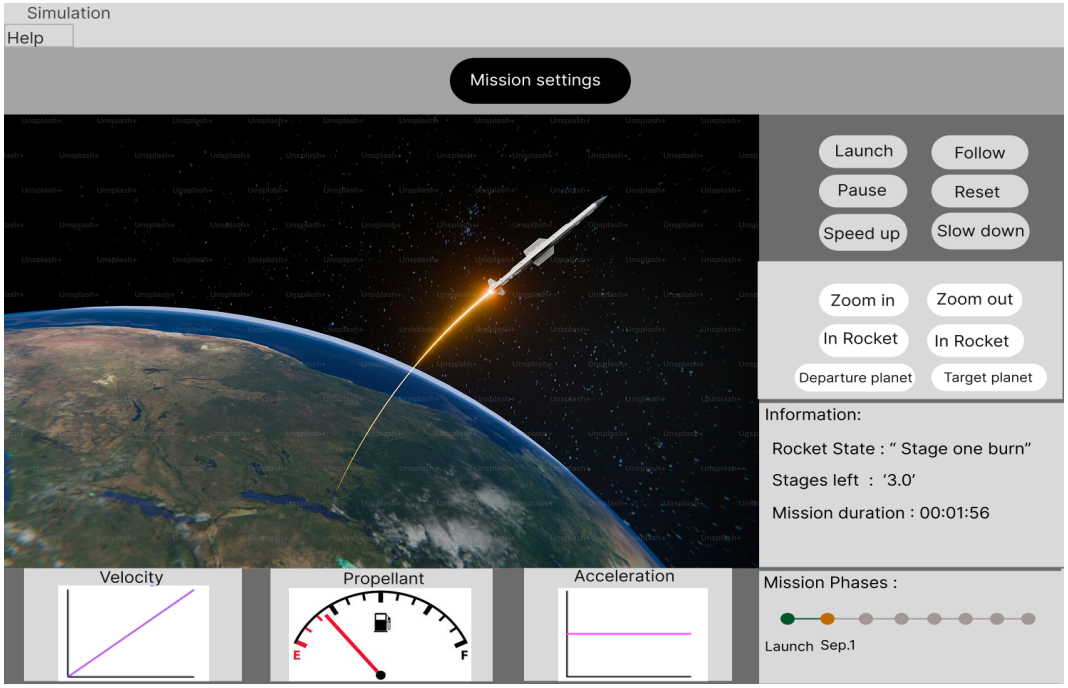


FIGURE 6 – Maquette de la fenêtre de simulation

l'application.

Mise en œuvre des managers. La méthode `nextRound()` de `SimulationManager` délègue à `MobileElementManager` selon le type de mission :

- `nextRoundLEO()` pour la mission orbite basse
- `nextRoundGEO()` pour la mission géostationnaire
- `nextRoundInterplanetary()` pour la mission interplanétaire

`MobileElementManager` implémente `MobileInterface` et constitue le cœur physique de la simulation. Il maintient l'état complet des objets mobiles à travers un ensemble de drapeaux booléens mis à jour au fil de la simulation : `isInOrbitLEO` indique si l'orbite basse a été atteinte, `isInOrbitGEO` si le satellite a rejoint l'orbite géostationnaire, `isInOrbitInter` si la sonde est en orbite autour de la planète cible, `isTransInjected` si l'injection interplanétaire a été déclenchée, et `failure` si la mission a échoué. Ces drapeaux conditionnent à chaque pas de temps les traitements à effectuer et garantissent que chaque phase de la mission ne s'exécute qu'une seule fois et dans le bon ordre.

Détail des méthodes de `MobileElementManager`.

- `rocketLaunch()`. Cette méthode est appelée une unique fois lors du clic sur le bouton Launch. Elle récupère le premier étage de la liste des étages de la fusée, active son moteur en positionnant le drapeau `inFire` à `true`, et définit cet étage comme étage courant. L'état de la fusée est positionné à `"Stage one burn"`.
- `updateRocketMass()`. Cette méthode calcule la masse totale de la fusée à chaque pas de temps. Elle commence par calculer le débit massique de l'étage actif à partir de sa poussée et de son impulsion spécifique selon l'équation 16, puis en déduit la nouvelle masse de propergol restant dans le réservoir. La masse totale de la fusée est ensuite recalculée en sommant les masses de tous les étages non encore séparés auxquels s'ajoute la masse de la charge utile si elle est toujours embarquée.

Algorithm 1 updateRocketMass()

Require: étage actif non nul, $\Delta t > 0$
Ensure: masse totale de la fusée mise à jour
  $F_p \leftarrow throttle \times thrust$
  $\dot{m} \leftarrow \dfrac{F_p}{I_{sp} \cdot g_0}$
  $m_{propergol} \leftarrow m_{propergol} - \dot{m} \cdot \Delta t$
  if $m_{propergol} < 0$ **then**
    $m_{propergol} \leftarrow 0$
  end if
  $m_{etage} \leftarrow m_{propergol} + m_{sec}$
  $m_{fusee} \leftarrow \sum_{etagesnonsepares} m_{etage} + m_{charge_utile}$

— `updateRocket()`. Cette méthode met à jour l'état physique complet de la fusée pour le pas de temps courant. Elle calcule l'altitude, détermine l'angle de virage gravitationnel, puis calcule successivement l'accélération, la vitesse et la position par intégration d'Euler.

Algorithm 2 updateRocket()

Require: fusée, planète, $\Delta t > 0$
Ensure: position, vitesse et altitude de la fusée mises à jour
  $d \leftarrow \sqrt{(\Delta x)^2 + (\Delta y)^2}$
  $h \leftarrow d - R_{planete}$
  if $h \leq 0$ **and** non en orbite **then**
    $failure \leftarrow$ **true**
  end if
  $\theta \leftarrow$ `calculateGravityTurn`(h)
  $(a_x, a_y) \leftarrow$ `calculateRocketAcceleration(fusee, planete)`
  $v_x \leftarrow v_x + a_x \cdot \Delta t$
  $v_y \leftarrow v_y + a_y \cdot \Delta t$
  $x \leftarrow x + v_x \cdot \Delta t$
  $y \leftarrow y + v_y \cdot \Delta t$

— `stageSeparation()`. Cette méthode vérifie à chaque pas de temps si les conditions de séparation de l'étage actif sont réunies. La séparation est déclenchée lorsque deux conditions sont simultanément satisfaites : le réservoir de l'étage est vide (masse de propergol nulle) et l'altitude de séparation prédéfinie de l'étage est atteinte. Lorsque ces conditions sont remplies, le moteur est éteint, l'étage est marqué comme séparé, sa masse est ramenée à sa masse à vide et le compteur d'étages restants est décrémenté.
— `updateStage()`. Cette méthode parcourt la liste des étages déjà séparés et met à jour leur position à chaque pas de temps. Les étages séparés ne sont plus propulsés : ils sont uniquement soumis à la gravitation de la planète, et leur trajectoire est calculée par `calculateStagePosition()`.

— `updateCurrentStage()`. Cette méthode gère la transition vers l'étage suivant après une séparation. Si des étages restent disponibles, le premier de la liste devient l'étage courant et son moteur est allumé. Si aucun étage ne reste, l'étage courant est positionné à `null`.

Algorithm 3 updateCurrentStage()

Require: étage courant
Ensure: activation de l'étage suivant après séparation
  if étage.séparé **and** moteur éteint **then**
    retirer étage de la liste active
    ajouter étage à la liste des séparés
    $H_{fusee} \leftarrow H_{fusee} - H_{etage}$
    if liste des étages non vide **then**
      étage courant ← étages[0]
      allumer moteur de l'étage courant
      mettre à jour l'état : `"Stage N burn"`
    else
      étage courant ← `null`
      mettre à jour l'état : `"All stages separated"`
    end if
  end if

— `payloadSeparation()`. Cette méthode déclenche le déploiement de la charge utile lorsque l'altitude de séparation prédéfinie est atteinte. Son comportement diffère selon le type de mission. Pour les missions LEO et GEO, chaque satellite est éjecté avec la vitesse orbitale circulaire calculée selon l'équation 12, appliquée dans la direction tangentielle. Pour la mission GEO spécifiquement, le satellite hérite directement de la vitesse de la fusée au moment de la séparation, car le dernier étage effectue lui-même la mise à vitesse orbitale. Pour la mission interplanétaire, la sonde est initialisée avec la vitesse orbitale autour de la Terre, puis sa planète cible et son orbite d'arrivée lui sont assignées depuis le système solaire.
— `updateSatellite()`. Cette méthode met à jour la position de chaque satellite déployé à chaque pas de temps, par délégation à `SimulationCalculation.calculateSatellitePosition()`. L'altitude du satellite est également recalculée à chaque itération.
— `updateProbe()`. Cette méthode met à jour la position de la sonde interplanétaire après son déploiement, par délégation à `SimulationCalculation.calculateProbePosition()`. Elle recalcule également l'altitude de la sonde par rapport à sa planète cible.
— `orbitInsertion()`. Cette méthode détecte la mise en orbite basse.

Algorithm 4 orbitInsertion()

Require: satellite ou sonde, planète, type de mission
Ensure: drapeau `isInOrbitLEO` levé si la mise en orbite est détectée
  $v_{orb} \leftarrow \sqrt{G \cdot M/d}$
  $v \leftarrow \sqrt{v_x^2 + v_y^2}$
  if mission LEO **or** mission GEO **then**
    if $v \geq 0{,}95 \cdot v_{orb}$ **and** $h > 200\,000$ **then**
      `isInOrbitLEO` ← **true**
      éteindre et séparer l'étage courant
    end if
  else
    if $|v - v_{orb}| < 0{,}05 \cdot v_{orb}$ **and** $h > 200\,000$ **then**
      `isInOrbitLEO` ← **true**
      éteindre et séparer l'étage courant
    end if
  end if

— `geoTransferInjection()`. Cette méthode est responsable de l'injection du satellite sur une trajectoire de transfert vers une orbite géostationnaire (GEO). Elle n'est exécutée que si la fusée est préalablement placée en orbite basse (LEO). Un délai, exprimé en nombre de frames, est imposé avant l'injection.

Algorithm 5 geoTransferInjection()

Require: fusée, planète, constantes physiques
Ensure: vitesse du satellite modifiée pour transfert GEO

if déjà en orbite GEO **then**
  return
end if
if pas en orbite LEO **then**
  return
end if
if aucun satellite **then**
  return
end if
$sat \leftarrow premier\ satellite$
$frameBeforeGeo \leftarrow frameBeforeGeo + 1$
if $frameBeforeGeo < FRAME_BEFORE_GEO$ **then**
  return
end if
$r_{LEO} \leftarrow distance(sat.position, planete.position)$
$r_{GEO} \leftarrow R_{planete} + ALTITUDE_{GEO}$
$v_{transfert} \leftarrow \sqrt{G \cdot M \cdot \left(\frac{2}{r_{LEO}} - \frac{1}{\frac{r_{LEO}+r_{GEO}}{2}}\right)}$
$(v_x, v_y) \leftarrow vitesse\ actuelle\ du\ satellite$
$v \leftarrow \sqrt{v_x^2 + v_y^2}$
$dir_x \leftarrow \frac{v_x}{v}$
$dir_y \leftarrow \frac{v_y}{v}$
$v_x \leftarrow v_{transfert} \cdot dir_x$
$v_y \leftarrow v_{transfert} \cdot dir_y$
$sat.vitesse \leftarrow (v_x, v_y)$
$isInOrbitGEO \leftarrow true$
état fusée ← "GEO transfer injection"

— `transInjection()`. Cette méthode est responsable de l'injection de la sonde sur une trajectoire de transfert vers une orbite interplanétaire. équivalente à `geoTransferInjection()`.

Algorithm 6 transInjection()

Require: fusée, planète, sonde, constantes physiques
Ensure: vitesse de la sonde modifiée pour transfert interplanétaire

if injection déjà effectuée **then**
  return
end if
if pas en orbite LEO **then**
  return
end if
if sonde nulle **then**
  return
end if
$frameBeforeTLI \leftarrow frameBeforeTLI + 1$
$frame \leftarrow FRAME_BEFORE_TLI$
if destination = Lune **then**
  $frame \leftarrow 2130$
end if
if destination = Mars **then**
  $frame \leftarrow 2400$
end if
if destination = Mercure **then**
  $frame \leftarrow 4528$
end if
if destination = Jupiter **then**
  $frame \leftarrow 1820$
end if
if $frameBeforeTLI < frame$ **then**
  return
end if
activer mode transfert de la sonde
$isTransInjected \leftarrow true$
$r_1 \leftarrow distance(sonde, plante)$
$r_2 \leftarrow distance(plante_cible, plante)$
$a \leftarrow \frac{r_1+r_2}{2}$
$v_{transfert} \leftarrow \sqrt{G \cdot M \cdot \left(\frac{2}{r_1} - \frac{1}{a}\right)}$
$(x, y) \leftarrow$ *position relative de la sonde*
$r \leftarrow \sqrt{x^2 + y^2}$
$t_x \leftarrow -\frac{y}{r}$
$t_y \leftarrow \frac{x}{r}$
$(v_x, v_y) \leftarrow$ *vitesse actuelle*
if $v_x \cdot t_x + v_y \cdot t_y < 0$ **then**
  inverser la tangente
end if
$v_x \leftarrow v_{transfert} \cdot t_x$
$v_y \leftarrow v_{transfert} \cdot t_y$
mettre à jour la vitesse de la sonde
état fusée ← "Trans Injection"

3.5 Design Patterns utilisés

La conception du simulateur repose sur l'utilisation de plusieurs design patterns classiques du génie logiciel.

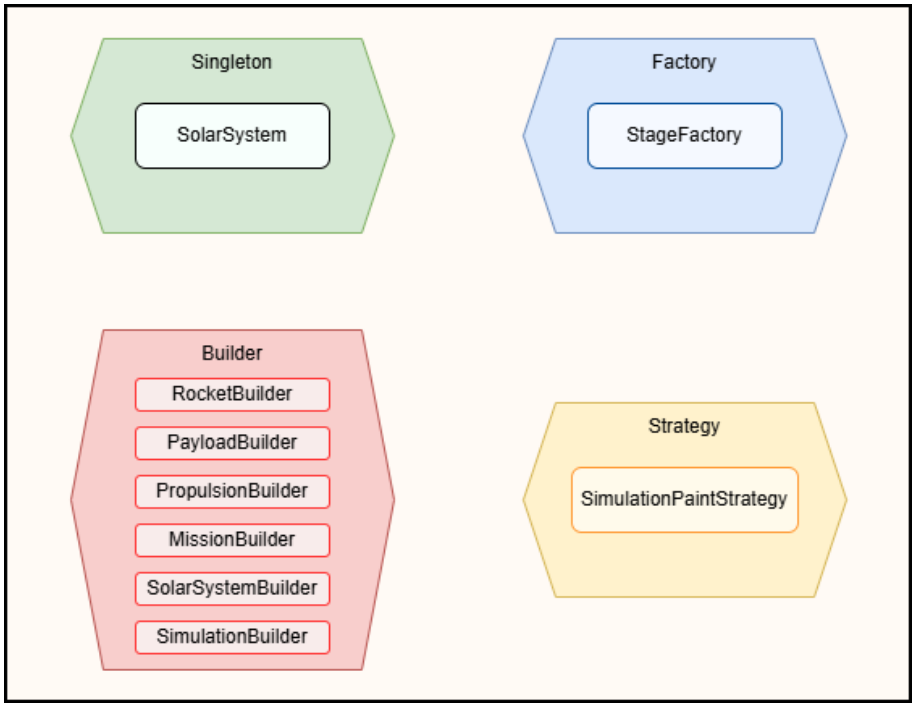


FIGURE 7 – Vue d'ensemble des patrons de conception utilisés

La figure 7 présente une vue d'ensemble des quatre patrons de conception mis en œuvre dans le simulateur.

Singleton pattern : `SolarSystem`. La classe `SolarSystem` : son constructeur est privé et une unique instance est créée de façon anticipée (`eager initialization`) via un attribut statique final. La méthode statique `getInstance()` permet à toutes les classes de l'application d'accéder à cette instance unique sans la recréer. Ce choix garantit que l'ensemble de la simulation partage un seul et même système solaire cohérent, évitant toute duplication des planètes et des orbites.

Builder pattern : La construction des objets complexes est déléguée à des classes Builder dédiées. `RocketBuilder` assemble les étages et la charge utile de la fusée selon le type de mission. `PropulsionBuilder` configure le moteur et le réservoir de chaque étage. `PayloadBuilder` construit la charge utile. `MissionBuilder` initialise les paramètres de la mission. `SolarSystemBuilder` instancie les planètes et les orbites. Enfin, `SimulationBuilder` appelle successivement chaque builder pour produire un `SimulationManager` complet et cohérent.

Factory pattern `StageFactory`. La création des étages de la fusée est centralisée dans la classe `StageFactory`, qui expose trois méthodes statiques : `createFirstStage()`, `createSecondStage()` et `createThirdStage()`. Chaque méthode calcule la position, la masse sèche, la masse de propergol et l'altitude de séparation propres à l'étage, selon le type de mission actif. Ce pattern isole la logique de création des étages et permet à `RocketBuilder` de s'appuyer sur une interface simple sans connaître les détails de construction.

Strategy pattern `SimulationPaintStrategy`. Le rendu graphique de la simulation est délégué à la classe `SimulationPaintStrategy`, séparée du panneau `SimulationDisplay` qui l'appelle. La stratégie de dessin est encapsulée dans une classe distincte, ce qui permet de modifier ou remplacer le rendu sans toucher au composant graphique principal.

4 Manuel utilisateur

Notre programme est organisé de sorte que l'utilisateur puisse aisément y naviguer et comprendre son fonctionnement, le but est que l'utilisateur est une approche pédagogique et peu abstraite des voyages spatiaux. Ainsi il sera en mesure de comprendre comment le lancement d'une fusée s'effectue, comment chaque parties de celle-ci fonctionnent et comment un satellite ou une sonde peut-être envoyé en orbite spatial. Notre application est donc divisé en deux parties.

4.1 Settings

La première page nommée "Settings" va permettre à l'utilisateur de paramétrer sa simulation, il décide du choix de la mission et de la destination vers quel endroit de l'espace voyager.

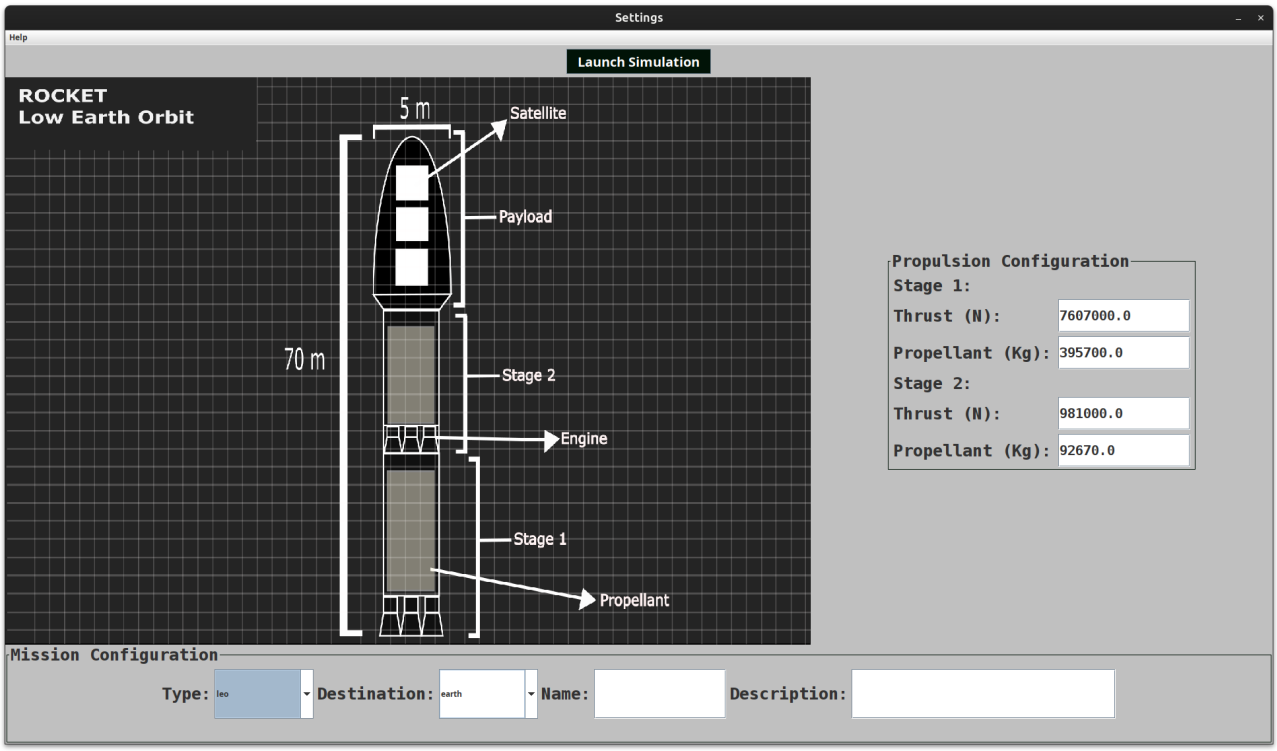


FIGURE 8 – Page d'ouverture de notre programme

La première page est divisée en quatre parties, celle du haut pour les boutons liés au menu, la partie gauche centrale pour afficher un aperçu de la fusée utilisée lors de la simulation, la partie droite pour montrer la configuration de la propulsion de la fusée et la dernière partie en bas rassemblant toutes les configurations de la mission que l'utilisateur veut mener.

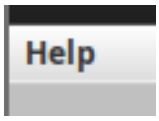


FIGURE 9 – Bouton "Help" dans la barre de menu des deux pages

Le bouton "Help" situé dans la barre de menu des deux pages, permet d'afficher toutes les informations nécessaire à la bonne compréhension de notre application et ainsi obtenir une description assez détaillée de chaque fonctionnalité de notre interface.

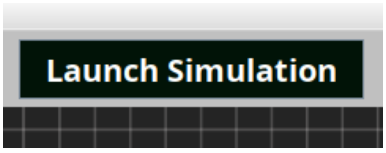


FIGURE 10 – Bouton "Launch Simulation" dans la barre de menu de "Settings"

Le bouton "Launch Simulation" dans le menu, permet de lancer la simulation selon les choix de l'utilisateur, ainsi il sera redirigé vers la page "Simulation" permettant de voir la mission spatiale se réaliser.

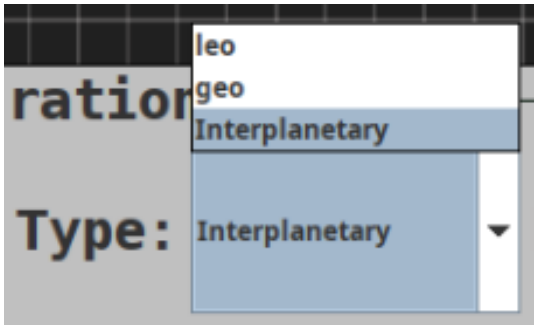


FIGURE 11 – Liste déroulante du type de mission

Ce combobox permet à l'utilisateur de choisir la mission qu'il veut effectuer, soit il envoie son engin spatial en orbite terrestre basse, soit en orbite géostationnaire, soit il envoie une sonde vers un autre astre.

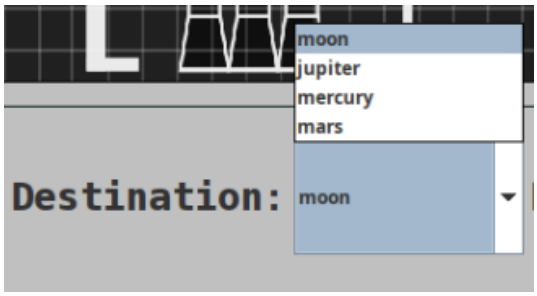


FIGURE 12 – Liste déroulante de la destination

Une liste déroulante permettant de choisir (si l'utilisateur a au préalable choisi la mission "interplanetary") l'astre spatial vers lequel il souhaite envoyer sa sonde soit vers la lune, Jupiter, Mars ou Mercure.



FIGURE 13 – Champ textuel personnalisable

Champ textuel du nom et de la description que l'utilisateur peut donner à sa mission de manière personnalisée.

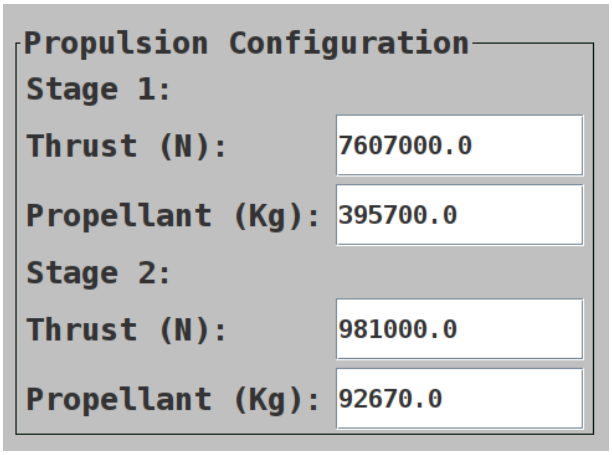


FIGURE 14 – Zone de texte récupérant les valeurs de propulsion

Champ textuel où la force de poussée et la masse de carburant de chaque étages peuvent être modifiées par l'utilisateur, sinon des valeurs par défaut correctes seront enregistrées pour le lancement de la fusée. Si l'utilisateur choisi la mission "Interplanetary" un troisième étage sera ajouté pour pouvoir modifier son contenu. A noter que le contenu des étages d'une fusée en orbite basse ou géostationnaire est bien différent d'une fusée interplanétaire.

4.2 Simulation

La seconde page nommée "Simulation" qui est la page principale de notre application,va permettre à l'utilisateur d'assister à la simulation du voyage spatial, il décide des paramètres en temps réels du déroulement de la simulation.

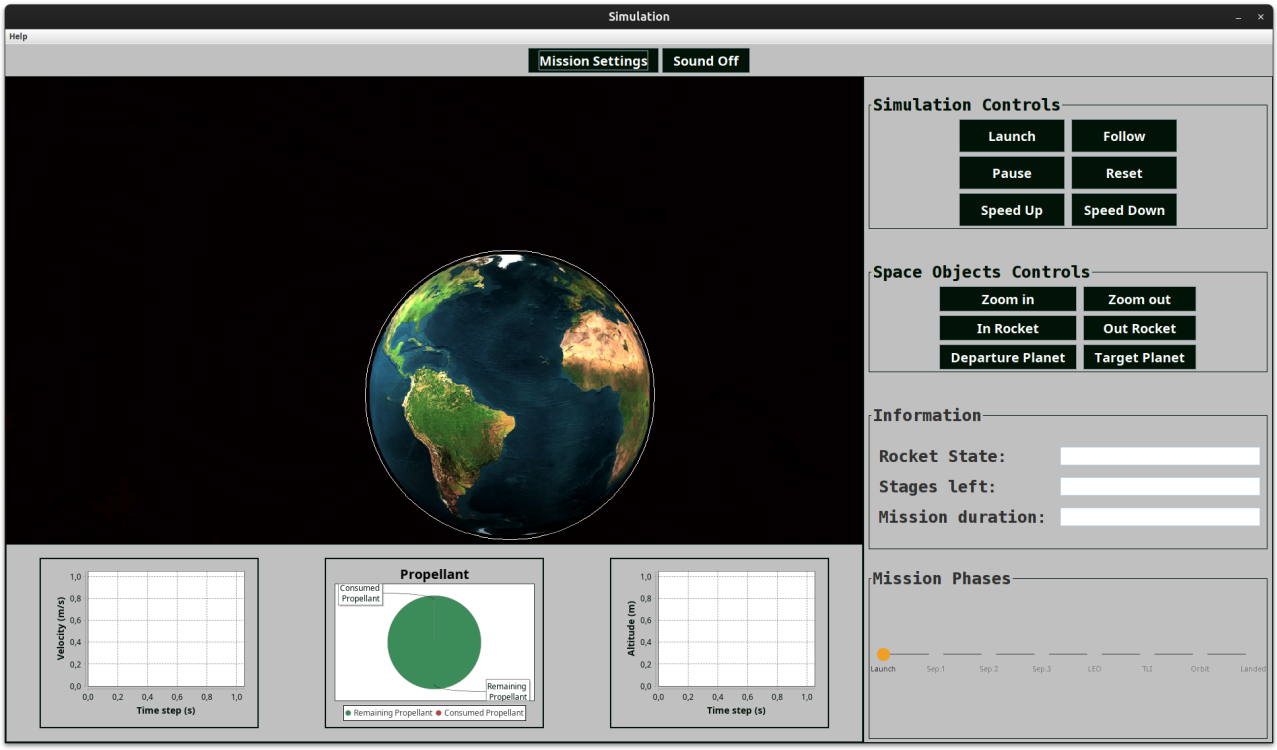


FIGURE 15 – Page principale du programme de la simulation

La deuxième page est divisée en quatres parties, celle du haut pour les boutons liés au menu, la partie gauche centrale pour afficher la simulation et observer l'avancement du voyage spatial de notre satellite ou de notre sonde jusqu'à son arrivée de destination, la partie droite constituée dans la première moitié

de boutons de controle de la simulation et dans la deuxième moitié des affichages en temps réel du comportement de la simulation. Pour finir la dernière partie en bas rassemblant tous les graphes de données physiques sur l'avancement de la fusée.



FIGURE 16 – Boutons du menu de la page "Simulation"

Le bouton "Mission Settings" permet de renvoyer l'utilisateur vers la première page pour le choix de la mission spatiale et ses configurations. Il y a aussi le bouton "Sound" permettant d'activer la musique pendant la simulation. A noter que le bouton "Help" reste le même mais que son contenu change selon page sur laquelle on se trouve.

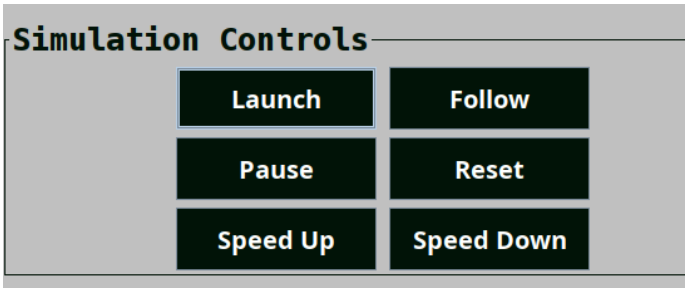


FIGURE 17 – Boutons de gestions de la simulation

Zone de boutons de controle de la simulation :
- `Launch` : Lancer la simulation et donc le décollage de la fusée
- `Follow` : Suivre la à la trace le satellite ou la sonde
- `Pause` : Arrêter temporairement la simulation
- `Reset` : Revenir au point de départ, avant que le lancement démarre
- `Speed Up` : Augmenter la vitesse d'affichage
- `Speed Down` : Diminuer la vitesse d'affichage

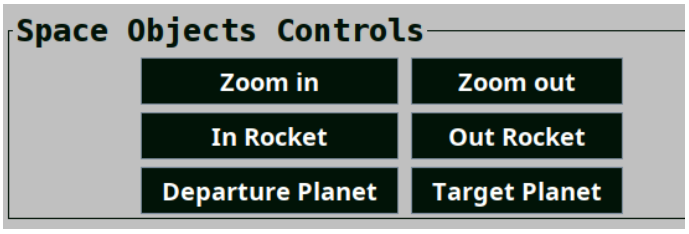


FIGURE 18 – Boutons de gestion de l'affichage

Zone de boutons de controle de l'affichage de la simulation :
- `Zoom in` : Se rapprocher de l'objet qu'on veut observer
- `Zoom out` : S'éloigner de l'objet qu'on veut observer
- `In Rocket` : Agrandir la fusée
- `Out Rocket` : Rétrécir la fusée
- `Departure Planet` : Focaliser l'écran sur la planète de départ
- `Target Planet` : Focaliser l'écran sur la planète d'arrivée

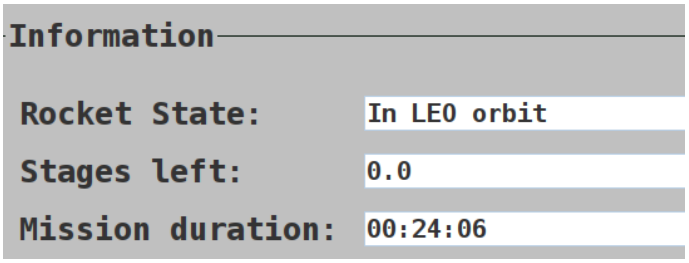


FIGURE 19 – Champ d'informations en temps réel de la simulation

Ce bloc d'information permet de communiquer trois informations sur la fusée en temps réel :

- `Rocket State` : L'état actuel de la fusée, si elle est en marche, ou qu'un problème est arrivé, on peut aussi voir ou se trouve la fusée.
- `Stages left` : Le nombre d'étages accrocher à la charge utile permettant de la propulsée dans l'espace.
- `Mission duration` : La durée de la simulation depuis le lancement de la fusée en heure,minute et seconde.

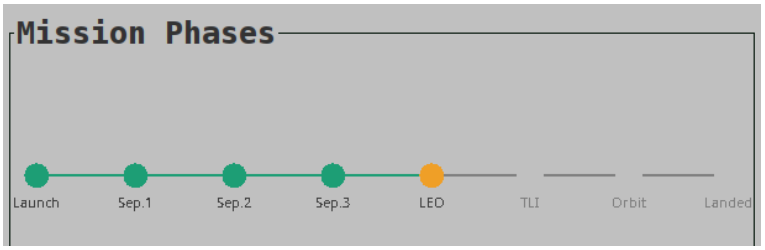


FIGURE 20 – Chaine d'indication de l'avancement de la simulation

Chaine représentant chaque étapes parcouru par la fusée depuis son lancment jusqu'à son arrivée.

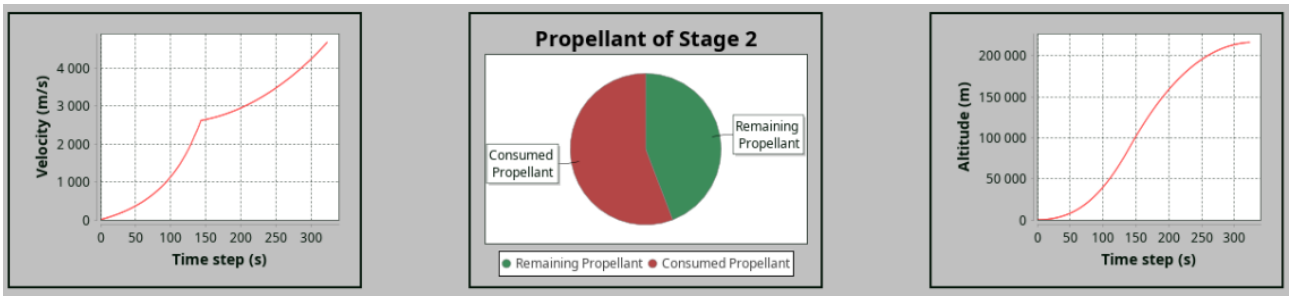


FIGURE 21 – Graphes des valeurs de vitesse,carburant et d'altitude de la fusée

Trois types de graphes :

- `Velocity` : la vitesse de la fusée
- `Propellant` : le carburant restant pour chaque étages de la fusée
- `Altitude` : la distance par rapport à une planète de départ ou d'arrivée

5 Déroulement du projet

Dans cette section, nous décrivons comment le projet a été réalisé en équipe : la répartition des tâches, la synchronisation du travail en membres de l'équipe, etc.

5.1 Réalisation du projet par étapes

Point de départ. Avant toute implémentation, une phase de recherche et de réflexion collective a été menée. Nous avons commencé par étudier les concepts de mécanique spatiale nécessaires à la simulation : les lois de Newton, la gravitation universelle, la traînée atmosphérique, le virage gravitationnel et les vitesses orbitales. Des ressources en ligne ont été consultées, notamment des articles scientifiques sur la reconstitution de trajectoires de fusées, des documentations de missions réelles de la NASA, ainsi que des ressources pédagogiques sur les calculs de trajectoire [tpe15]. Cette phase nous a également permis de partager nos idées sur l'architecture du logiciel, de définir les classes de données principales et de nous répartir les premières tâches de développement avant d'entamer le codage. Une fois ces bases établies, le développement s'est organisé en quatre étapes successives, comme illustré dans la figure 22.

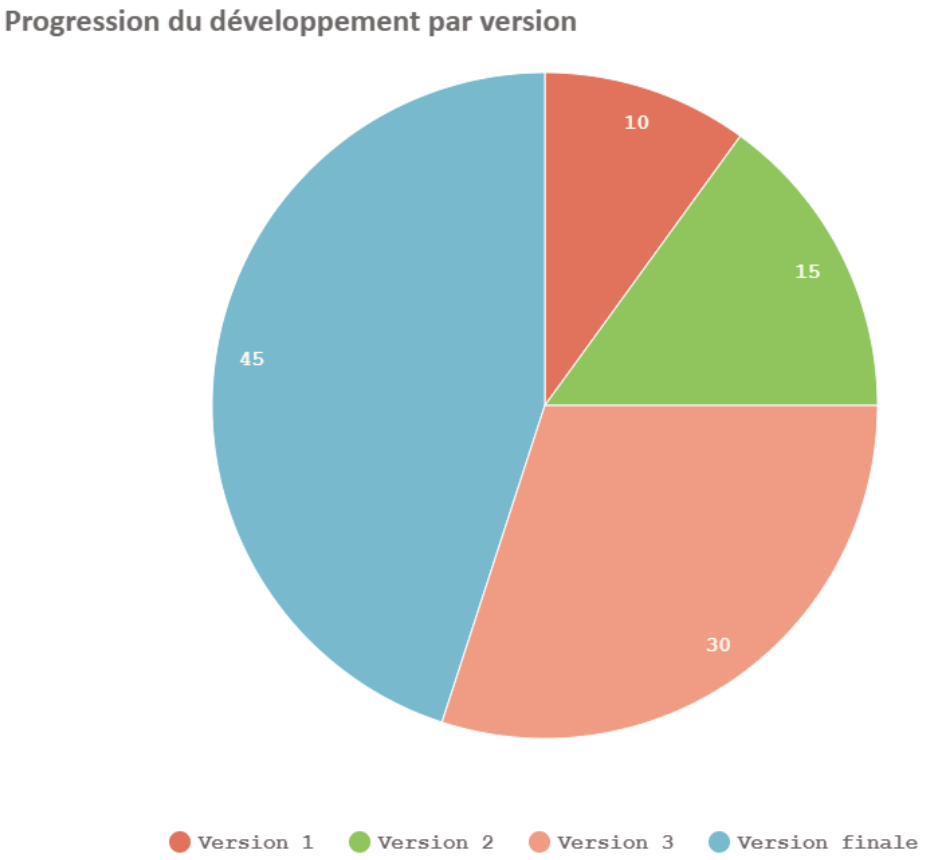


FIGURE 22 – Répartition de l'avancement du projet par version

Version 1 (10%) : La première version avait pour objectif de poser les bases de l'application : définir l'architecture générale, les classes de données et un premier affichage graphique. Le mouvement de la fusée était simulé de façon simplifiée, sans échelle réelle ni physique newtonienne. La séparation du premier étage était déclenchée lorsque la fusée atteignait une altitude seuil prédéfinie. La mise en orbite était approximative : le satellite était placé sur une trajectoire circulaire calculée géométriquement, sans tenir compte des forces gravitationnelles.

Version 2 (15%) : La deuxième version a constitué une amélioration majeure du moteur physique. La physique newtonienne a été pleinement intégrée :`SimulationCalculation` regroupe désormais l'ensemble des calculs réels de mécanique spatiale, notamment le calcul de l'accélération en tenant compte des trois forces (poussée, gravitation universelle et traînée atmosphérique), l'intégration d'Euler pour la vitesse et la position, ainsi que le calcul du débit massique et de la consommation de carburant à chaque pas de temps. La gestion des étages a également été refaite :`MobileElementManager` distingue désormais clairement `updateRocketMass()`,`updateRocket()`, `stageSeparation()` et`updateCurrentStage()` en méthodes séparées et spécialisées. Le virage gravitationnel est calculé dynamiquement en fonction de l'altitude via `calculateGravityTurn()`. La détection de mise en orbite basse repose sur une comparaison avec l'altitude LEO réelle. Enfin, la gestion des échecs de mission a été introduite via le drapeau `failure`.

Version 3 (30%) : La troisième version a représenté une refonte complète et l'aboutissement du simulateur. Les trois types de missions ont été pleinement implémentés : LEO, GEO et interplanétaire. `MobileElementManager` a été enrichi avec les méthodes `nextRoundLEO()`, `nextRoundGEO()` et `nextRoundInterplanetary()`, chacune gérant les phases spécifiques à sa mission. La mission GEO intègre le transfert de Hohmann via `geoTransferInjection()` et la circularisation de l'orbite via `geoInsertion()`. La mission interplanétaire introduit l'injection trans-planétaire, la capture orbitale autour de la planète cible et l'atterrissage via `landing()`. Les builders ont été finalisés : `RocketBuilder`, `PropulsionBuilder`,`PayloadBuilder`, `MissionBuilder`,`SolarSystemBuilder` et `SimulationBuilder` produisent un graphe d'objets complet et cohérent. L'interface graphique a été complétée avec tous ses panneaux fonctionnels, les graphiques temps réel de vitesse, carburant et accélération.

Version finale (45%) : La version finale a été consacrée aux finitions et à la stabilisation de l'ensemble du simulateur. Le système solaire complet a été intégré avec la possibilité de choisir la planète de destination parmi plusieurs options.L'interface graphique a été peaufinée, la `SimulationPaintStrategy` optimisée pour le rendu des planètes, orbites et trajectoires.Les valeurs physiques ont été calibrées et validées pour garantir le réalisme de chaque type de mission.

En parallèle du développement, chaque séance de TD donnait lieu à un rendu documentaire complémentaire.Le TD 1 portait sur la rédaction du document de spécification du projet (cahier de charges).Le TD 2 concernait la conception des classes de données (UML).Le TD 3 portait sur la conception de l'IHM graphique.Le TD 4 concernait la conception du noyau fonctionnel.Le TD 5 portait sur les tests du logiciel.Enfin, le TD 6 concernait la mise en place d'un système de journal et d'instrumentation du logiciel. Ces rendus intermédiaires ont permis de structurer le développement et de valider progressivement les choix de conception.

5.2 Répartition des tâches, communication et intégration

Tâche	**Responsable**
Modèle de données	
Classes de données, classes mobiles, classes cinématiques	Lydia, Thomas, Sabri
Physique / Calculs	
SimulationCalculation	Lydia
MobileElementManager mission LEO	Lydia
MobileElementManager mission GEO	Thomas
MobileElementManager mission Interplanétaire	Lydia, Thomas
Design Patterns	
Singleton, Builder, Factory, Strategy	Lydia
Interface graphique	
EntryGUI	Lydia, Thomas
MainGUI	Lydia
SimulationDisplay, SimulationPaintStrategy	Lydia
FocusManager, ChartManager, ChartsData	Lydia
Panneaux IHM (PropulsionPanel, ChartsPanel...)	Lydia
Utilitaires	
EngineConfig, PhysicsConstants, SimulationConfig	Lydia, Thomas
LoggerUtility	Lydia
Rapport	Lydia, Thomas

Table 3 – Répartition des tâches entre les membres de l'équipe

Le tableau 3 présente la répartition des tâches entre les membres de l'équipe. Le travail a débuté collectivement par la conception du modèle de données, à laquelle les trois membres ont contribué. Par la suite, le développement s'est organisé de façon asymétrique : Lydia a pris en charge le moteur physique, les design patterns, l'ensemble de l'interface graphique et les utilitaires. Thomas a contribué au développement de la mission GEO ainsi qu'à la fenêtre de configuration et à la rédaction du rapport. La communication au sein de l'équipe s'est appuyée sur des échanges réguliers lors des séances de TD, complétés par des échanges en dehors des séances pour synchroniser l'avancement et résoudre les difficultés rencontrées. L'outil utilisé pour le partage et la gestion du code est **GitHub** : chaque membre soumettait ses contributions sur un dépôt commun, permettant de suivre l'historique des modifications et d'intégrer le travail de chacun de façon progressive et contrôlée au fil des versions.

6 Conclusion et perspectives

Dans cette section, nous résumons la réalisation du projet et nous présentons également les extensions et améliorations possibles du projet.

6.1 Résumé du travail réalisé

Ce projet nous a permis de concevoir et d'implémenter un simulateur pédagogique de missions spatiales en Java. L'application finale couvre l'ensemble du cycle d'une traversée spatiale, depuis la configuration de la fusée jusqu'au déploiement de la charge utile en orbite ou vers une planète cible.

Le simulateur intègre un moteur physique reposant sur les lois de la mécanique newtonienne. Les trois forces appliquées à chaque pas de temps, à savoir la poussée des moteurs, la gravitation universelle et les frottements de l'air, permettent de calculer des trajectoires réalistes. Le virage gravitationnel est modélisé dynamiquement en fonction de l'altitude, et la consommation de carburant est calculée à chaque seconde selon l'impulsion spécifique de chaque moteur.

Les trois types de missions ont été pleinement implémentées. La mission LEO permet la mise en orbite basse d'un satellite à 200 km d'altitude. La mission GEO réalise un transfert de Hohmann depuis l'orbite basse jusqu'à l'orbite géostationnaire à 35 786 km, avec une première impulsion en LEO et une circularisation à l'apogée. La mission interplanétaire envoie une sonde vers la Lune via une injection trans-planétaire suivie d'une capture orbitale autour de la planète cible.

Sur le plan logiciel, le projet applique les bonnes pratiques de conception orientée objet étudiées en cours. Quatre patrons de conception ont été mis en œuvre : le Singleton pour le système solaire, le Builder pour la construction des objets complexes, la Factory pour la création des étages et la Strategy pour le rendu graphique. L'architecture est organisée en quatre modules fonctionnels aux responsabilités bien séparées : configuration, données, traitement et interface graphique.

L'interface graphique offre à l'utilisateur une expérience complète. La fenêtre de configuration permet de paramétrer la mission, de modifier les valeurs de propulsion et de visualiser le schéma de la fusée. La fenêtre de simulation affiche l'animation spatiale en temps réel, accompagnée de graphiques de vitesse, de carburant et d'altitude, ainsi que d'un panneau d'informations sur l'état de la fusée et l'avancement de la mission.

6.2 Améliorations possibles du projet

Plusieurs pistes d'amélioration peuvent être envisagées pour enrichir le simulateur dans de futures versions.

Amélioration du réalisme physique. Il serait très interessant de pouvoir réaliser la vraie rotation des planètes du système solaire pour obtenir des trajectoires bien plus réalistes et adaptatives au mouvment des astres. De la même manière on pourrait rajouter une page dédiée uniqument au mouvment du système solaire pour admirer le mouvment des planètes. Il serait aussi intéressant de pouvoir laisser libre court au demandes de l'utilisateur quand il veut choisir une orbite, qu'il n'ai pas le choix entre seulement deux orbites mais qu'il puisse par exemple choisir ses valeurs de carburant ensuite le logiciel envoie la fusée selon l'orbite atteignable par les données de l'utilisateur, ainsi il n'y aurait pas deux orbites terrestre mais tous les orbites possible entre 200km et 35000km.

Extension des missions disponibles. Le simulateur pourrait être enrichi avec de nouveaux types de missions, comme un retour sur Terre depuis l'orbite, ou encore une mission de déploiement de plusieurs satellites, on pourrait aussi rajouter une dimension économique à notre projet tel que le fait de récupérer des matériaux dans l'espace, les ramener sur terre puis les revendre pour acheter de meilleurs équipements, ces matériaux pourraient être récoltés depuis les astéroides ou des exos planètes.

Amélioration de l'interface graphique. L'affichage pourrait être enrichi par une représentation plus détaillée de la fusée pendant le vol, avec une visualisation des flammes des moteurs et des animations de séparation des étages. L'ajout d'une vue miniature globale permettant de situer la fusée par rapport à l'ensemble du système solaire améliorerait la lisibilité de la simulation lors des missions interplanétaires. Il faudrait rendre l'expèrience encore plus immersive en modélisant un astronaute dans son cokpit choisissant ses missions, ici l'astronaute serait l'utlisateur, et pourquoi pas le faire déplacer dans tout le vaisseau pour voir comment un humain vie dans l'espace sans gravité. Il serait aussi très interessant de modéliser le décollage au lancment de la fusée (en gros plan sur la fusée sur terre) ou même modéliser en gros plan l'atterissage d'une sonde sur une autre planète.

Gestion des pannes et des échecs. Le simulateur pourrait intégrer des scénarios de pannes moteur ou de défaillance structurelle, permettant à l'utilisateur de comprendre les conséquences de paramètres de propulsion inadaptés. Un mode d'analyse post-mission, affichant un bilan complet de la trajectoire, des vitesses atteintes et des phases accomplies, constituerait également un ajout pédagogique pertinent.

Références

[CNE24] CNES. Les fusées, comment ça marche ? https://cnes.fr/dossiers/fusees, 2024. Consulté le 26 avril 2026.

[ker24] Gravity turn. https://wiki.kerbalspaceprogram.com/wiki/Gravity_turn/fr, 2024.

[LV05] João Emile Louis and Nandamudi L. Vijaykumar. On calculating and visualizing rocket trajectory reconstitution. *IAE - CTA / INPE*, 2005.

[NAS69] NASA. Mission Apollo 11. Mission spatiale, July 1969.

[NAS11] NASA. Mars Science Laboratory. Mission spatiale, November 2011.

[tpe15] Les calculs de trajectoire d'une fusée. https://tpe-la-conquete-de-l-espace.webnode.fr/notre-projet/les-deplacement-dans-lespace/les-calculs-de-trajectoire-dune-fusee/, 2015.

[Wik26] Wikipédia. Orbite de transfert. https://fr.wikipedia.org/wiki/Orbite_de_transfert, 2026.

[you21] Rocket trajectory simulation. https://www.youtube.com/watch?v=RnkXL0qzb9U, 2021.